

Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Computer Architecture and Assembly Lab (14:332:333)
Fall 2024

Lab 5 *RISC-V Leaf Procedures & Stacks*

Goals

In Lab 5, we will start from Lab 4, and continue to discuss the following topics:

- Stack in RISC-V programming.
- Leaf procedures.

Instructions

- All the programs must be compiled by Venus RISC-V Simulator. Please note that the simulator only supports 32-bit instruction, and we do not consider overflow.
- Once you complete the assignment, please upload your source code and PDF lab report on Canvas.
- **Comment your code clearly. You can use the mark "#" to add comments.**
- You can copy and paste your code into the lab report.

Introduction

Venus Simulator Review

Environmental call (ecall)

Similar to "printf" in C, ecall is to show the content in the console. Register `x10` stores the function code, and register `x11` stores the content that we want to show.

Assembler directives

Besides instructions in assembler format, an assembler also accepts so-called assembler directives.

- `.text`: Where the code starts. Each assembler instruction can start with a label such as `main:`.
- `.data`: data segment, where you can initialize data. Also, data can be addressed by using a label.



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

RISC-V Calling Conventions in Register Usages

In RISC-V programming, the calling convention is a standard that defines how functions receive parameters and return values. It also determines which registers are used for what purpose and who (caller or callee) is responsible for saving and restoring the contents of registers.

ra (x1): return address. No need to save & restore
a0-a7 (x10–x17): registers for procedures
a0, a1 (x10, x11): function arguments/return values
a2-a7 (x12–x17): function arguments. No need to save & restore
t0-t6 (x5–x7, x28–x31): temporary registers. Not preserved by the callee. No need to save & restore
s0-s11 (x8, x9, x18–x27): saved registers. Must be preserved by the callee. Need to save & restore

Introduction to RISC-V Functions/Procedures

Function/Procedures is a stored subroutine that performs a specific task based on the provided parameters.
Procedure call: jump-and-link, e.g., `jal ra, ProcedureLabel`

- Puts the address of the following instruction to **ra**
- Jumps to target address **ProcedureLabel**

Procedure return: jump-and-link register, e.g., `jalr x0, 0(ra)`

- Like **jal**, but jumps to (0 + address in **ra**)
- Use **ra** as the destination register (rd) (**x0** cannot be changed).

Steps required:

1. Place parameters in registers (**a0** to **a7**)
2. Transfer control to procedure
3. Acquire storage resources needed for procedure
4. Perform procedure's operations
5. Place results in registers for the caller (**a0**, **a1**)
6. Return control to the point of origin (address in **ra**)

Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

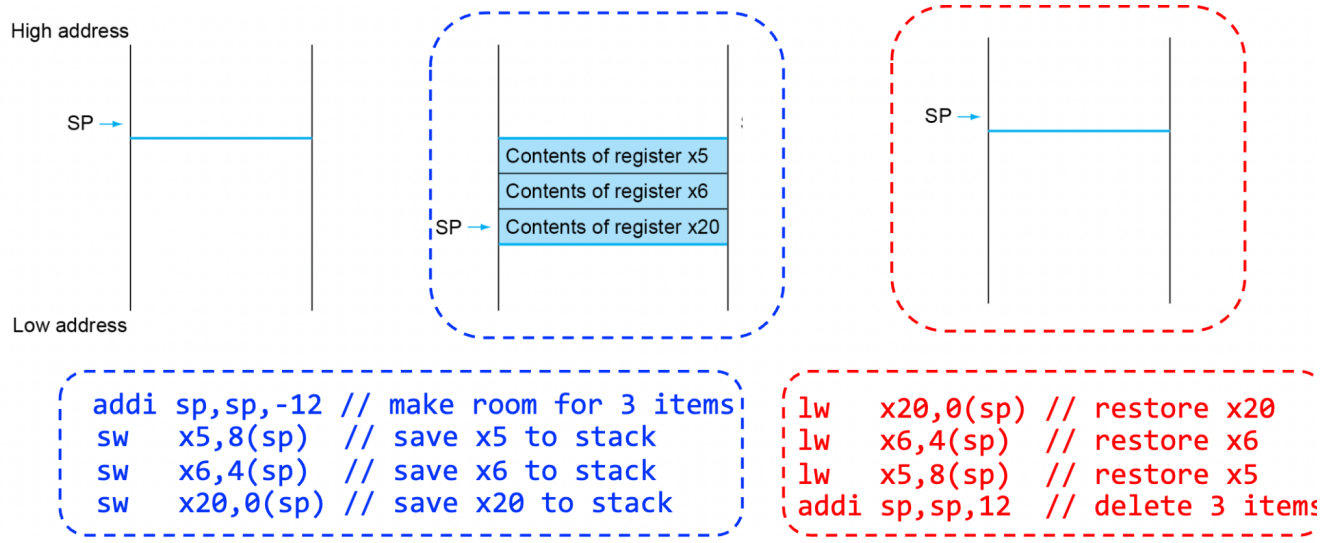
Stack in RISC-V

Scenario: need more registers than **x10** to **x17**

Problem: values of additional registers would change

Stack: spill registers to memory

sp: stack pointer



Example:

C code:

```
int foo(int g, int h, int i, int j) {
    int f;
    f = (g + h) - (i + j);
    return f;
}
int main() {
    int s = 0;
    s = foo(3, 4, 5, 6);
    return 0;
}
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

The corresponding RISC-V code:

The foo procedure:

```
foo:
addi sp,sp,-4  # make room for 1 item
sw   x20,0(sp)  # save x20 to stack
add  x5,x10,x11 # x5 = g + h
add  x6,x12,x13  # x6 = i+ j
sub  x20,x5,x6  # x20 = x5-x6
addi x10,x20,0  # x10 = x20
lw   x20,0(sp)  # restore x20 from stack
addi sp,sp,4   # delete 1 item
jalr x0,0(x1)   # return to the caller
```

The caller:

```
addi x18,x0,0 # s = 0
addi x10,x0,3 # x10 = 3
addi x11,x0,4 # x11 = 4
addi x12,x0,5 # x12 = 5
addi x13,x0,6 # x13 = 6
jal  x1, foo  # call procedure foo
addi x18,x10,0 # s = x10
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Exercise 1 RISC-V Procedures and Stacks [25 pts]

1.1 [5 pts] What is **`sp`** and how should it be used in the context of RISC-V functions?

Sp is a dedicated register used for the stack pointer, and it holds the address of the item at the top of the stack. In the context of RISC-V functions, it's used to allocate and deallocate memory.

1.2 [20 pts] Please Write a RISC-V program that performs operations with values stored in the stack. Your program should accomplish the following steps:

1. We can assume that three integers **A**, **B**, and **C** are available in registers **`x10`**, **`x11`**, and **`x12`**, respectively. You can give any values to registers **`x10`**, **`x11`**, and **`x12`**.
2. Push **A**, **B**, and **C** into the stack using the stack pointer (**`sp`**) register.
3. Pop **B** and **C**, compute the sum of **B** and **C**, and store the sum (**B+C**) into the register **`x10`**.
4. Pop **A**, compute the product of the sum (**B+C**) and **A**, and push the result into the stack.
5. Finally, pop the computed result (**A*(B+C)**) from the stack and also print it using **`ecall`**.

In your report, please include the following:
The screenshots of **your program's output** and the **stack memory during steps 2, 3, and 4**.



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

169

Registers Memory Cache VDB

Address	+3	+2	+1	+0
0x7FFFFFF0	00	00	00	00
0x7FFFFFEC	00	63	2E	68
0x7FFFFFE8	00	00	00	00
0x7FFFFFE4	7F	FF	FF	EC
0x7FFFFFE0	00	00	00	0D
0x7FFFFFDC	00	00	00	A9
0x7FFFFFD8	00	00	00	03
0x7FFFFFD4	00	00	00	00
0x7FFFFFD0	00	00	00	00
0x7FFFFFCC	00	00	00	00
0x7FFFFFC8	00	00	00	00
0x7FFFFFC4	00	00	00	00
0x7FFFFFC0	00	00	00	00

Jump to -- choose -- Up Down



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

(1)

(2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**

```
.data

result: .word 0


.text

.globl main

main:

   addi x10, x0, 3    # a = 3

   addi x11, x0, 8   # b = 8

   addi x12, x0, 13   # c = 13

   addi sp, sp, -12   # allocate space for 3 integers

   sw x10, 0(sp)  # push A

   sw x11, 4(sp)  # push B

   sw x12, 8(sp)   # push C

   lw x11, 4(sp)  # load B from stack

   lw x12, 8(sp)   # load C from stack

   add x10, x11, x12  # b+c and store in x10

   addi sp, sp, 8  # deallocate B and C

   lw x10, 0(sp)   # load A from stack
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
mul x11, x10, x10  # (B+C)*A

addi sp, sp, -4   # allocate space for the result

sw x11, 0(sp)    # push the result

lw x11, 0(sp)    # load the result from stack

addi sp, sp, 4     # deallocate the result

li a0, 1

ecall

li a0, 10
ecall
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Exercise 2 Leaf & Non-leaf Procedures [75 pts]

2.1 [20 pts] Please write a RISC-V program in Venus simulator that accepts an input integer x and computes the following function:

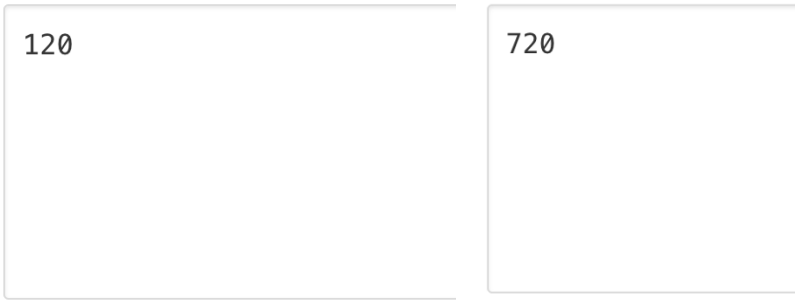
$$f(x) = x! = x * (x-1) * (x-2) *.....* 2 * 1$$

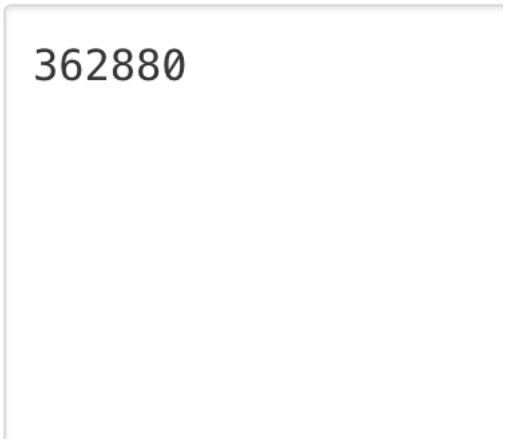
You can assume x is no bigger than 10.
In the program, please have a main procedure that takes the value of the input, performs the factorial computations, and prints the output in the console.

Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

In your report, please include the following:

(1) A screenshot of your program's output. **Verify your program with 3 different input values x_1, x_2, and x_3 such that $x_1 \neq x_2 \neq x_3$.**





(2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**

```
.data

result: .word 0

.text

.globl main

main:
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
    addi x10, x0, 9
    addi x11, x0, 1          # result = 1
loop:
    beq x10, x0, exit        # if x == 0, exit
    mul x11, x11, x10        # result = result * x
    addi x10, x10, -1        # x = x - 1
    j loop                   # jump back to the start of the loop
exit:
    addi sp, sp, -4          # allocate space on stack for the result
    sw x11, 0(sp)            # store the result on the stack
    lw x11, 0(sp)            # load the result back into x11
    addi sp, sp, 4           # deallocate the stack space
    li a0, 1
    ecall
    li a0, 10
    ecall
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

2.2 [20 pts] Please write a RISC-V program that when given an integer (32-bit) array **`arr = {2, 6, 10, 14, 20, 16}`** in **`.data`** directive, returns the array in **reverse order** (i.e. the first number is now the last one). You can assume that the array always contains an even number of elements. Your code should include the following procedures:

Write a **`main`** procedure to perform the following tasks:

1. Call an **`input`** procedure to write the given array values into a certain continuous memory starting from **`0(0x10000040)`**.
2. Call a **`reorder`** procedure to reorder the array.
3. Call an **`output`** procedure to print the reordered array values in the console.



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

In your report, please include the following:



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

(1) A screenshot of the memory (starting from **`0x10000040`**).



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

Registers Memory Cache VDB

Address	+3	+2	+1	+0
0x10000058	00	00	00	00
0x10000054	00	00	00	02
0x10000050	00	00	00	06
0x1000004C	00	00	00	0A
0x10000048	00	00	00	0E
0x10000044	00	00	00	14
0x10000040	00	00	00	10
0x1000003C	00	00	00	00
0x10000038	00	00	00	00
0x10000034	00	00	00	00
0x10000030	00	00	00	00
0x1000002C	00	00	00	00
0x10000028	00	00	00	00

Jump to -- choose -- Up Down

Address: Go



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

(2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**

```
.data
    arr: .word 2, 6, 10, 14, 20, 16
    newline: .string "\n"
    space: .string " "

.text
.globl main

input:
    li x10, 0x10000040     # destination address
    la x11, arr            # source address
    li x12, 6              # counter

input_loop:
    beq x12, zero, input_done
    lw x20, 0(x11)
    sw x20, 0(x10)
    addi x10, x10, 4
    addi x11, x11, 4
```




```
    addi x12, x12, -1
    beq zero, zero, input_loop
input_done:
    jalr x0, x1, 0

reorder:
    mul x20, x11, x8      # x8 is 4
    add x21, x10, x20     # End address
    addi x21, x21, -4     # Last element address

reorder_loop:
    bge x10, x21, reorder_done
    lw x22, 0(x10)        # load first element
    lw x23, 0(x21)        # load last element
    sw x23, 0(x10)        # store last at first
    sw x22, 0(x21)        # store first at last
    addi x10, x10, 4      # move front pointer
    addi x21, x21, -4     # move back pointer
    beq zero, zero, reorder_loop
reorder_done:
    jalr x0, x1, 0
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
output:
    addi x20, x10, 0      # save start address
    addi x21, x11, 0      # save size

output_loop:
    beq x21, zero, output_done
    lw x11, 0(x20)        # load number in a1 for print
    li x10, 1             # print integer syscall
    ecall
    la x11, space         # print space
    li x10, 4
    ecall
    addi x20, x20, 4
    addi x21, x21, -1
    beq zero, zero, output_loop
output_done:
    la x11, newline
    li x10, 4
    ecall
    jalr x0, x1, 0

main:
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
li x8, 4              # constant 4 for array element size
jal input
li x10, 0x10000040    # array address
li x11, 6             # array size
jal reorder
li x10, 0x10000040    # array address
li x11, 6             # array size
jal output
li x10, 10
ecall
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

2.3 [35 pts] Write a RISC-V program in the Venus simulator that computes matrix multiplication. You can assume all the entries in each matrix are positive numbers. In the program, you need to consider the following cases:

- When the number of rows for the matrix A and the number of columns for the matrix B are equal, compute the multiplication and obtain the product matrix C: $C = A * B$.
- When the number of rows for the matrix A and the number of columns for the matrix B are not equal, return the error code 99. Hint: please refer to the Ecall wiki page about returning the error code (https://github.com/kvakil/venus/wiki/Environmental-Calls).

In the program, please have a main function that checks the condition, performs the appropriate computations, and prints the matrix C or the error code in the console.

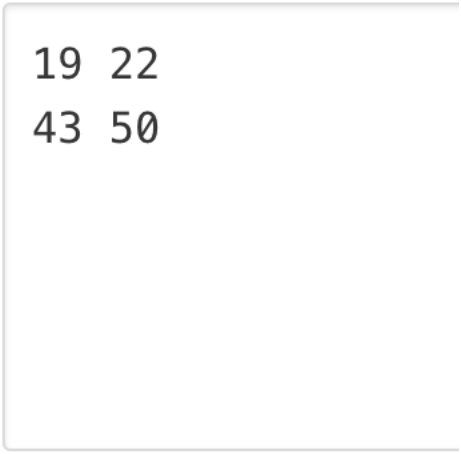
- Verify your program for 3 input matrices $m_1, m_2,$ and m_3 such that the dimensions of $m_1, m_2,$ and m_3 are all unique.
- Demonstrate each possible case at least once.
- Your implementation should work for matrices of different values as well.



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

In your report, please include the following:

(1) A screenshot of your program's output.



(2) Your annotated program code, with comments for clarity and understanding. **Please copy and paste the code into your report.**

```
.data
    matrixA: .word 1, 2    # First row
             .word 3, 4    # Second row
    rowsA: .word 2
    colsA: .word 2
    matrixB: .word 5, 6    # First row
             .word 7, 8    # Second row
    rowsB: .word 2
    colsB: .word 2
    matrixC: .space 16     # 4 elements * 4 bytes each

    newline: .string "\n"
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
    space: .string " "


.text

.globl main


main:

    la x5, rowsA

    lw x20, 0(x5)        # x20 = rows of A

    la x5, colsA

    lw x21, 0(x5)        # x21 = cols of A

    la x5, rowsB

    lw x22, 0(x5)        # x22 = rows of B

    la x5, colsB

    lw x23, 0(x5)        # x23 = cols of B

    li x24, 0           # i = 0 (row counter for A)

outer_loop:

    bge x24, x20, done    # if i >= rowsA, done

    li x25, 0           # j = 0 (col counter for B)

middle_loop:

    bge x25, x23, outer_next # if j >= colsB, next row

    li x26, 0           # Sum for this element

    li x27, 0           # k = 0 (element counter)

inner_loop:

    bge x27, x21, middle_next # if k >= colsA, store result

    # Calculate indices

    # A[i][k] = matrixA + 4*(i*colsA + k)
```



Department of Electrical and Computer Engineering
Rutgers, The State University of New Jersey

```
    mul x5, x24, x21     # i * colsA
    add x5, x5, x27      # + k
    slli x5, x5, 2       # * 4
    la x6, matrixA
    add x6, x6, x5
    lw x7, 0(x6)         # Load A[i][k]
    # B[k][j] = matrixB + 4*(k*colsB + j)
    mul x5, x27, x23     # k * colsB
    add x5, x5, x25      # + j
    slli x5, x5, 2       # * 4
    la x6, matrixB
    add x6, x6, x5
    lw x8, 0(x6)         # Load B[k][j]
    # Multiply and add
    mul x9, x7, x8
    add x26, x26, x9     # sum += A[i][k] * B[k][j]
    addi x27, x27, 1     # k++
    beq zero, zero, inner_loop
middle_next:
    # Store result in C[i][j]
    mul x5, x24, x23     # i * colsC
    add x5, x5, x25      # + j
    slli x5, x5, 2       # * 4
    la x6, matrixC
    add x6, x6, x5
    sw x26, 0(x6)        # Store result
```




```
    addi x25, x25, 1     # j++
    beq zero, zero, middle_loop
outer_next:
    addi x24, x24, 1     # i++
    beq zero, zero, outer_loop
done:
    li x24, 0            # Row counter
print_outer:
    bge x24, x20, exit   # if row >= rowsA, exit
    li x25, 0            # Column counter
print_inner:
    bge x25, x23, print_next # if col >= colsB, next row
    mul x5, x24, x23
    add x5, x5, x25
    slli x5, x5, 2
    la x6, matrixC
    add x6, x6, x5
    lw x11, 0(x6)
    li x10, 1
    ecall
    li x10, 4
    la x11, space
    ecall
    addi x25, x25, 1     # col++
    beq zero, zero, print_inner
print_next:
```

```
    li x10, 4
    la x11, newline
    ecall
    addi x24, x24, 1     # row++
    beq zero, zero, print_outer
exit:
    li x10, 10
    ecall
```